

ATTENTIONPAY

Interpretable Transformer Model for
Explainable Fraud Detection

SOFTWARE DESIGN DOCUMENT

Final Consolidated Architecture, Implementation, Dataset, Inference and
Explainability Specification

Document Version	1.0
Status	Final Project SDD
Date	29 September 2026
Project Type	Academic payment-security simulation
Primary AI Models	BERT + TabTransformer
Backend	FastAPI + PostgreSQL
Frontend	React + Vite
Explainability	BERT URL explanation + SHAP/LIME

Document Purpose
This document defines the final AttentionPay software architecture and implementation contract. It consolidates the project objectives, layered security workflow, BERT phishing detection, PostgreSQL-backed transaction simulation, deterministic security validation, canonical 11-feature TabTransformer pipeline, SHAP/LIME explainability, FastAPI interfaces, React frontend behavior, data design, testing strategy, and operational setup.

Prepared as the repository-level technical design document for the AttentionPay project.

Document Control and Design Baseline

This SDD represents the consolidated final project design. It describes the implemented project behavior and the finalized model/inference contract as a single technical baseline.

Item	Final Baseline
Project name	AttentionPay: Interpretable Transformer Model for Explainable Fraud Detection
Security stages	Stage 1 — phishing URL security; Stage 2 — transaction security and behavioural fraud detection
Stage 1 model	Trained BERT Transformer URL classifier
Stage 2 deterministic checks	API Route Integrity and Impossible Travel
Stage 2 AI model	TabTransformer behavioural fraud classifier
Model feature contract	11 features; fixed order
Label mapping	0 = Fraud; 1 = Legitimate
Operating threshold	0.332
Training source	200,000-row raw canonical fraud dataset
Split	Stratified 70/15/15 train/validation/test
Class distribution	45% Fraud / 55% Legitimate
XAI	BERT URL explanation; backend rule explanations; SHAP/LIME for TabTransformer
Simulation modes	Normal Transaction; Impossible Travel; API Route Tampering; Behaviour Fraud; Random Transaction

Design principle: the backend is the source of truth. Scenario selection determines which controlled transaction is retrieved; it does not determine the final prediction. The frontend visualizes the actual returned execution path and does not fabricate model outcomes or explanation values.

Table of Contents

Section	Page
1. Project Overview	3
2. System Requirements	4
3. Overall System Architecture	5
4. Technology Stack and Deployment Context	7
5. Stage 1 — BERT Phishing URL Detection	8
6. Stage 2 — Controlled Transaction Simulation	9
7. Transaction Scenarios and Simulation Modes	10
8. Layer 1 — Deterministic Backend Security	11
9. Feature Engineering and Canonical 11-Feature Contract	12
10. Dataset Preparation and Training	13
11. Stage 2 — TabTransformer Fraud Detection	15
12. Explainable AI Architecture	16
13. Frontend Design and Implementation	18
14. Frontend ↔ Backend Interaction	20
15. Backend Design — FastAPI	21
16. Database Design	23
17. Complete System Workflow	25
18. Result Dashboard Logic	26
19. Repository Structure	27
20. Implementation and Integration Workflow	28
21. Testing Strategy	29
22. Expected Outputs, Limitations and Future Enhancements	31
23. Conclusion	33

1. Project Overview

1.1 Project Purpose

AttentionPay is a layered payment-security simulation designed to address two connected security problems: phishing attacks that occur before a transaction and suspicious transaction behaviour that occurs during the transaction workflow. The system combines a trained BERT URL classifier, deterministic backend security gates, a TabTransformer fraud model, and decision-specific explainability.

1.2 Problem Statement

Online payment fraud can involve more than the final transaction record. A user may first be redirected through a phishing website, a request may be routed through an unexpected API endpoint, the transaction may originate from an impossible location transition, or the behavioural pattern may differ from the expected user context. AttentionPay therefore evaluates risk progressively and only executes the AI fraud stage when the transaction is eligible.

1.3 Objectives

- Detect phishing payment URLs using a trained BERT Transformer model.
- Prevent a user from proceeding to the transaction simulator when the submitted URL is classified as phishing.
- Provide a safe controlled payment simulation without real banking infrastructure or real money movement.
- Retrieve controlled demonstration transactions from PostgreSQL according to the selected simulation mode.
- Apply deterministic API Route Integrity and Impossible Travel checks before AI fraud detection.
- Generate the canonical 11-feature model vector for transactions that pass the backend security gates.
- Detect behavioural fraud using a trained TabTransformer.
- Provide explanations appropriate to the layer that produced the decision.
- Expose the actual processing path through a React dashboard.

1.4 Scope

The project covers phishing URL analysis, transaction simulation, backend security validation, behavioural fraud classification, explainability, database-backed scenario retrieval, and frontend visualization. It is an academic and demonstration system rather than a production banking platform.

1.5 Core Design Principles

- Layered security: earlier deterministic gates can stop processing before the AI stage.
- Backend source of truth: final status, predictions, probabilities, checks, and explanations come from backend responses.
- Frozen model contract: the same feature order and preprocessing artifacts are used for training, testing, and inference.
- Decision-specific explanations: a backend rule block is not represented as an AI model decision.
- Controlled simulation: transaction values originate from stored demo rows rather than frontend-generated outcomes.
- No fabricated state: the UI never invents a prediction, feature value, explanation, or execution state.

2. System Requirements

2.1 Functional Requirements

ID	Requirement
FR-01	Accept a payment URL and send it to the BERT analysis service.
FR-02	Return phishing/legitimate prediction, confidence/probabilities, blocked state, and URL explanation when applicable.
FR-03	Allow Stage 2 only after a legitimate Stage 1 result.
FR-04	Provide five Stage 2 simulation modes.
FR-05	Retrieve a controlled transaction from PostgreSQL for the selected mode.
FR-06	Display the retrieved transaction as read-only data.
FR-07	Run API Route Integrity and Impossible Travel checks.
FR-08	Block and explain transactions that fail Layer 1.
FR-09	Prepare the exact 11-feature model vector for eligible transactions.
FR-10	Run TabTransformer inference and return prediction/probabilities.
FR-11	Generate SHAP/LIME explanations for AI-executed transactions.
FR-12	Render pipeline states according to actual backend execution.
FR-13	Allow Random Transaction runs to be repeated against fresh database rows.
FR-14	Maintain temporary Stage 2 access authorization between stages.

2.2 Non-Functional Requirements

- Consistency: training and inference must use identical feature ordering and preprocessing.
- Integrity: scenario labels must never be treated as predictions.
- Explainability: every final decision must identify the layer/source responsible for it.
- Security: Stage 2 requests are protected by a temporary access token issued after a legitimate URL result.
- Maintainability: model, feature engineering, database, API, and explanation logic remain modular.
- Responsiveness: the frontend presents loading/processing states without claiming streaming execution.
- Reproducibility: model artifacts and preprocessing artifacts are saved and reused for inference.

2.3 Security and Safety Requirements

- No real payment credentials, OTPs, PINs, or monetary transfers are required by the system.
- Transaction data displayed by the simulator is controlled demonstration data.
- Stage 2 access is not manually entered by the user.
- A 403 response clears Stage 2 session state and requires returning to Stage 1.
- Explanation data must correspond to the actual decision source.

3. Overall System Architecture

3.1 Layered Architecture

AttentionPay is organized into two principal security stages. Stage 1 evaluates the payment URL. Stage 2 evaluates a controlled transaction through deterministic security checks and, when eligible, the TabTransformer.

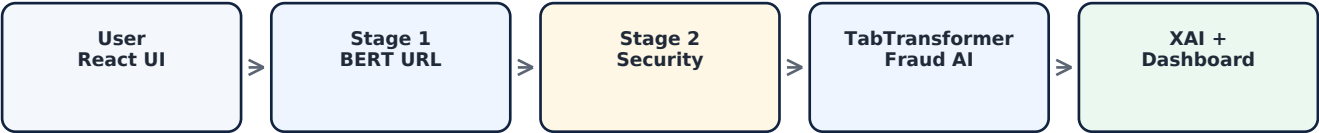


Figure 3.1 — High-level AttentionPay security architecture

3.2 End-to-End Architecture

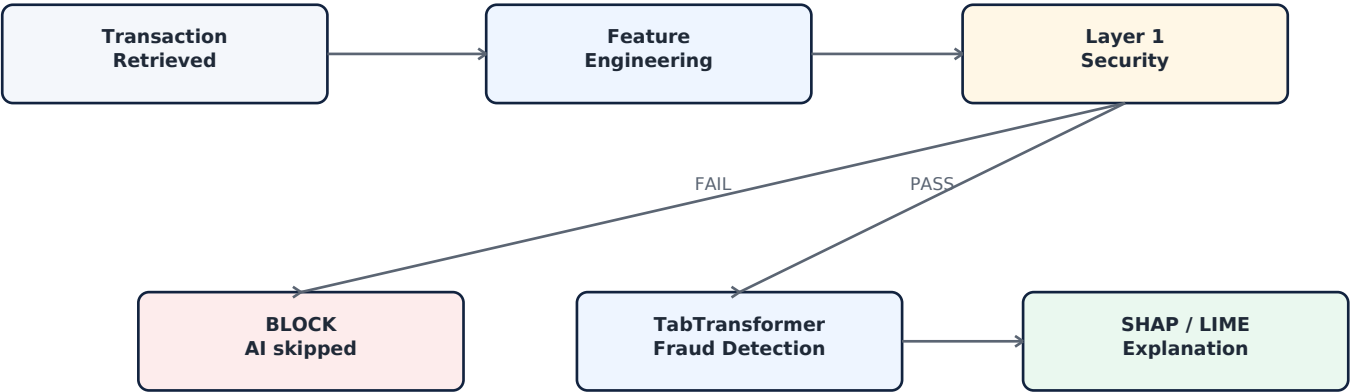


Figure 3.2 — Stage 2 processing and decision branches

3.3 Architectural Principle

Stage 2 is a database-backed transaction simulator. Scenario selection controls which stored transaction is retrieved, but the selected scenario is not itself the final prediction. The actual result is determined by backend security checks and, where eligible, the TabTransformer output.

3.4 Processing Boundaries

Layer	Responsibility	Decision Source
Stage 1	Phishing URL classification	BERT
Layer 1	API Route Integrity + Impossible Travel	Deterministic backend rules
Layer 2	Behavioural fraud classification	TabTransformer
XAI	Decision explanation	BERT explanation / backend rules / SHAP-LIME
Presentation	Visualize actual returned state	React frontend

4. Technology Stack and Deployment Context

Area	Technology	Role
Frontend	React + Vite	Interactive security session and result dashboard
HTTP client	Axios	Frontend ↔ backend communication
Visualization	Recharts, React Leaflet	Probability/contribution charts and location visualization
Backend	FastAPI + Uvicorn	API routing and orchestration
Validation/config	Pydantic Settings, python-dotenv	Configuration and request validation
ORM/database	SQLAlchemy + PostgreSQL	Controlled transaction persistence and retrieval
Stage 1 ML	Hugging Face Transformers + PyTorch	BERT URL inference
Stage 2 ML	PyTorch / TabTransformer	Tabular behavioural fraud inference
Data processing	Pandas, NumPy, Scikit-learn	Preparation, preprocessing and evaluation
Explainability	SHAP + LIME	Feature-level model explanation
Artifacts	Joblib + PyTorch checkpoints	Preprocessing and model persistence

4.1 Runtime Context

The backend runs as a FastAPI application and exposes Swagger/OpenAPI documentation. The frontend is a Vite development application. PostgreSQL provides the controlled transaction data source. Trained BERT and TabTransformer artifacts are loaded by backend services.

4.2 Local Startup

```
python -m venv .venv
.venv\Scripts\activate
pip install -r backend/requirements.txt
uvicorn backend.app.main:app --reload
```

The backend is available locally at <http://127.0.0.1:8000> and its interactive API documentation is available at [/docs](#).

4.3 Frontend Startup

```
cd frontend
npm install
npm run dev
```

4.4 Configuration

Environment-specific settings, including database connection information and the frontend origin, are supplied through backend configuration. Secrets and credentials must not be committed to the repository.

5. Stage 1 — BERT Phishing URL Detection

5.1 Purpose

Stage 1 is the first security gate. It determines whether the payment URL is legitimate before the transaction simulator is exposed.

5.2 Model

A trained BERT Transformer model is stored with its configuration and tokenizer artifacts under the repository's model/ directory. The backend BERT service is responsible for model loading, tokenization, inference, and confidence generation.

5.3 Input and Output

Direction	Data
Input	Payment or website URL entered by the user
Output	Prediction, confidence/probabilities, blocked/allowed state
Phishing explanation	Highlighted URL/token components and optional attention information
Legitimate result	Stage 2 access token enabling the next stage

5.4 Stage 1 Flow

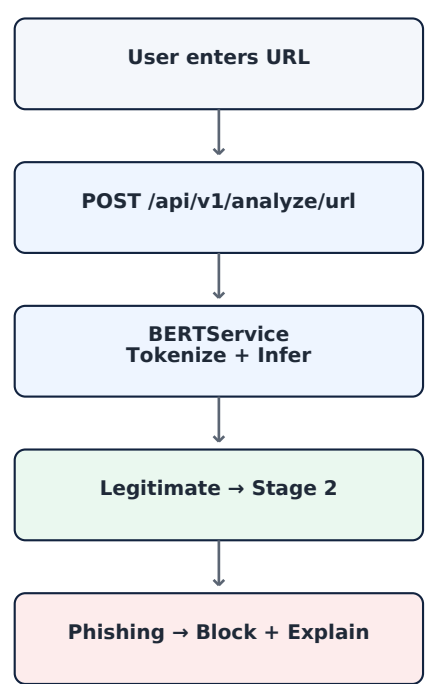


Figure 5.1 — Stage 1 URL analysis flow

5.5 Explanation Integrity

The URL explanation highlights components the model focused on. Attention or token importance must not be described as guaranteed causal proof. A legitimate URL proceeds without requiring a phishing explanation.

5.6 API Contract

POST /api/v1/analyze/url

Request: {"url": "https://example.com"}

Response fields: prediction, confidence, phishing_probability,
legitimate_probability, blocked, explanation_source?, explanation?,
stage2_access_token?

6. Stage 2 — Controlled Transaction Simulation

6.1 Purpose

The Stage 2 simulator provides a controlled environment in which stored demonstration transactions can be processed through the same security and AI path.

6.2 Simulation Workflow

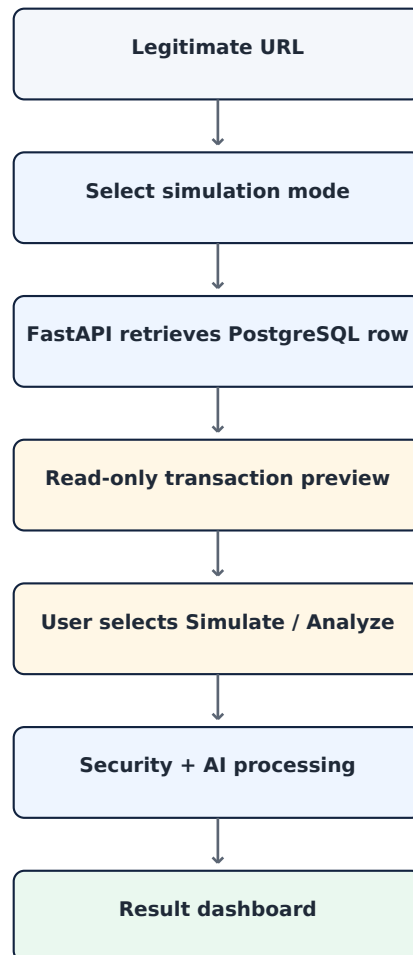


Figure 6.1 — Transaction simulation lifecycle

6.3 Transaction Retrieval

For a named scenario, the backend selects a matching stored transaction row. For Random Transaction, any row from the complete demonstration transaction pool is eligible. The selected row is returned to the frontend and displayed read-only.

6.4 Random Transaction

Random Transaction is an integration mode rather than a prediction mode. The backend selects a stored row, the row passes through the common security pipeline, and the final result depends on the actual row and the actual processing path. A repeated run requests a fresh selection.

6.5 Stage 2 Access Control

After a legitimate Stage 1 result, the backend can issue a temporary Stage 2 access token. The frontend stores it in session state and attaches it to Stage 2 requests through the X-Stage2-Access-Token header. The token is not intended to be manually entered or displayed.

7. Transaction Scenarios and Simulation Modes

Mode	Database selection	Expected processing path
Normal Transaction	Matching normal_transaction row	Backend checks pass → TabTransformer → XAI → result
Impossible Travel	Matching impossible_travel row	Travel rule fails → Block → rule/map explanation → AI skipped
API Route Tampering	Matching api_route_tampering row	Endpoint mismatch → Block → rule explanation → AI skipped
Behaviour Fraud	Matching behaviour_fraud row	Backend checks pass → TabTransformer evaluates behaviour → XAI
Random Transaction	Any demo row	Common pipeline; result depends on actual selected row

7.1 Scenario Metadata vs Prediction

The scenario identifier is used for database organization and retrieval. It is not the AI prediction. A row selected from the Behaviour Fraud scenario is still passed to the TabTransformer and receives an actual model prediction.

7.2 Database Selection Rules

```
Named scenario:
SELECT * FROM demo_transactions
WHERE scenario_id = :scenario_id
ORDER BY RANDOM() LIMIT 1;
```

```
Random Transaction:
SELECT * FROM demo_transactions
ORDER BY RANDOM() LIMIT 1;
```

7.3 Demonstration Data Principles

- The selected database row is the source of transaction values shown in the UI.
- The frontend does not generate or modify model feature values.
- Repeated Random Transaction runs can select different transaction IDs.
- The internal scenario label is not displayed as a live prediction.

8. Layer 1 — Deterministic Backend Security

8.1 Purpose

Layer 1 performs security checks before the AI fraud stage. These checks are deterministic backend rules and are deliberately separated from the TabTransformer model input.

8.2 Impossible Travel

The backend receives previous/current latitude and longitude together with previous/current transaction timestamps. It calculates distance, elapsed time, and the required travel speed. If the required speed exceeds the configured project threshold, the transaction is blocked.

8.3 Impossible Travel Flow

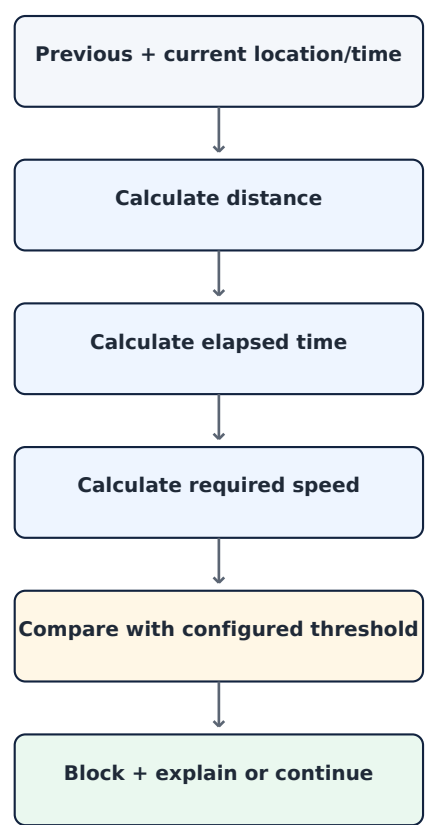


Figure 8.1 — Impossible Travel decision flow

8.4 API Route Integrity

The backend compares the expected payment endpoint with the actual endpoint. A mismatch indicates API route tampering and causes the transaction to be blocked/held.

8.5 Layer 1 Output Contract

Condition	Backend result
Rule failure	backend_status=blocked; ai_executed=false; final_status=blocked; explanation source identifies th
Rules pass	backend_status=passed; transaction becomes eligible for feature engineering and TabTransformer

8.6 Separation from ML

Impossible Travel and API Route Integrity are not TabTransformer features. They are pre-model security gates. This ensures that an infrastructure/location rule decision is not incorrectly attributed to the behavioural ML classifier.

9. Feature Engineering and Canonical 11-Feature Contract

9.1 Purpose

Feature engineering converts an eligible transaction into the exact input contract expected by the trained TabTransformer. The order is frozen across dataset preparation, training, validation, testing, and inference.

#	Feature	Type / meaning
1	known_device_flag	Binary; known device = 1, otherwise 0
2	device_changed_flag	Binary; changed device context = 1, otherwise 0
3	device_type	Categorical: desktop / mobile / tablet
4	browser_name	Categorical: chrome / firefox / edge / safari
5	operating_system	Categorical: android / ios / windows / macos / linux
6	transactions_last_1min	Numerical transaction count in previous 1 minute
7	transactions_last_5min	Numerical transaction count in previous 5 minutes
8	transactions_last_10min	Numerical transaction count in previous 10 minutes
9	transaction_amount	Numerical current transaction amount
10	previous_transaction_amount	Numerical previous transaction amount
11	session_risk_score	Numerical current session behavioural risk score

9.2 Feature Contract Rules

- Categorical vocabularies are fixed to the documented values.
- Binary flags use 0/1 consistently.
- $\text{transactions_last_1min} \leq \text{transactions_last_5min} \leq \text{transactions_last_10min}$.
- Historical features must use information available before the current transaction.
- No feature may be generated from the target label.
- Numerical preprocessing is applied using artifacts fitted only on the training split.

9.3 Inference Processing

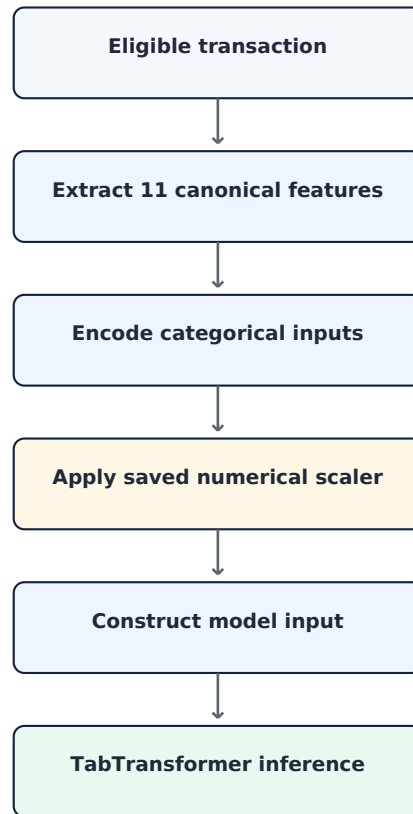


Figure 9.1 — Canonical feature engineering and inference flow

10. Dataset Preparation and Training

10.1 Canonical Training Dataset

The canonical training source contains 200,000 raw transaction rows and 12 columns: 11 model features plus the target label. The raw dataset is used as the training source so that numerical preprocessing can be fitted only on the training split.

Property	Value
Rows	200,000
Columns	12 total: 11 features + label
Fraud	90,000 rows (45%), label 0
Legitimate	110,000 rows (55%), label 1
Missing values	None in the audited raw dataset
Exact duplicates	None in the audited raw dataset
Target mapping	0 = Fraud; 1 = Legitimate

10.2 Train / Validation / Test Split

Split	Rows	Fraud	Legitimate
Train	140,000	63,000	77,000
Validation	30,000	13,500	16,500
Test	30,000	13,500	16,500
Total	200,000	90,000	110,000

The split is stratified 70/15/15 and preserves the 45/55 class distribution. A fixed random seed is used for reproducibility.

10.3 Preprocessing

- Separate X and y before preprocessing.
- Do not scale categorical features.
- Fit the numerical scaler only on the training split.
- Transform validation, test, and inference using the same fitted scaler.
- Persist preprocessing artifacts for backend inference.

10.4 Training Flow

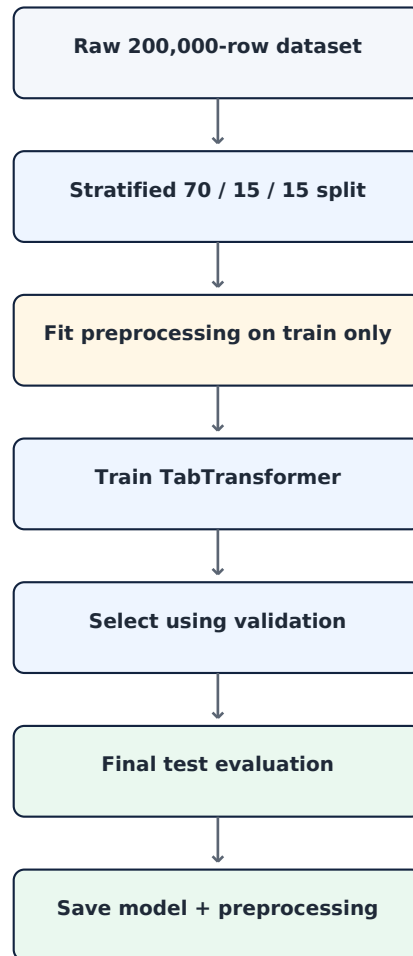


Figure 10.1 — Dataset preparation and training workflow

10.5 Training Contract

The TabTransformer uses 3 categorical inputs and 8 numerical inputs corresponding to the canonical 11-feature order. The test split remains untouched until final evaluation.

11. Stage 2 — TabTransformer Fraud Detection

11.1 Purpose

The TabTransformer is the second-stage behavioural classifier. It evaluates transactions that have already passed the deterministic backend security gates.

11.2 Model Contract

Property	Final value
Model family	TabTransformer
Input features	11
Categorical inputs	3 categorical inputs plus project categorical/binary preprocessing
Numerical inputs	8 numerical inputs
Label 0	Fraud
Label 1	Legitimate
Operating threshold	0.332
Inference artifacts	model checkpoint, categorical encoders, scaler, feature configuration

11.3 Evaluation Protocol

The evaluation protocol includes Accuracy, Precision, Recall, F1-score, ROC-AUC, PR-AUC, confusion matrix inspection, threshold analysis, and false-positive/false-negative inspection.

11.4 Selected Model Evaluation Baseline

Metric	Value
Accuracy	72.76%
Precision	72.61%
Recall	81.02%
F1-score	76.59%
ROC-AUC	80.00%
PR-AUC	81.77%
Operating threshold	0.332
Fraud precision	72.98%
Fraud recall	62.65%
Fraud F1-score	67.42%

11.5 Confusion Matrix

Actual \ Predicted	Fraud	Legitimate
Fraud	8,458	5,042
Legitimate	3,131	13,369

11.6 Inference Output

- Fraud/Legitimate prediction.
- Fraud and legitimate probabilities.
- Decision according to the operating threshold.
- The exact feature vector used for inference when AI executes.

- XAI payload generated from the same inference contract.

12. Explainable AI Architecture

12.1 Purpose

AttentionPay uses different explanation mechanisms for different decision sources. This prevents the system from presenting a model explanation when a deterministic backend rule actually caused the block.

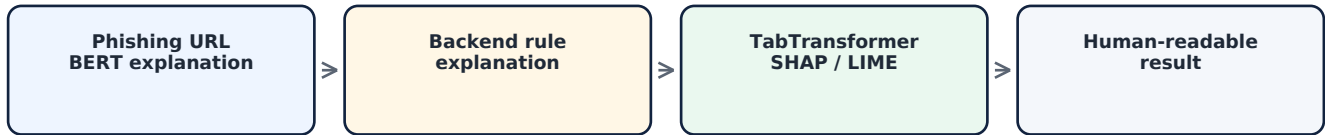


Figure 12.1 — Decision-specific explainability architecture

12.2 Stage 1 URL Explanation

- Uses BERT-oriented URL/token explanation.
- Returns suspicious token components and importance/position information when available.
- Optional attention information can be shown as model-focused evidence, not causal proof.

12.3 Layer 1 Rule Explanation

- Impossible Travel explanation includes locations, timestamps, distance, elapsed time, required speed and threshold context.
- API Route Integrity explanation compares expected and actual endpoints.
- The explanation source identifies the backend rule engine.
- SHAP/LIME are not executed when Layer 1 blocks the transaction.

12.4 TabTransformer XAI

The Stage 2 explanation layer uses the same 11-feature contract, categorical encoders, scaler, feature ordering, model, and a real training-data background for SHAP/LIME explanation. The explanation must correspond to the exact model input used for prediction.

XAI output	Purpose
summary	Human-readable explanation summary
confidence_text	Prediction confidence wording
key_factors	Main influential factors
features	Shared feature list with value and SHAP/LIME contribution
shap	Feature-level SHAP contribution values
lime	Feature-level LIME weights
interpretation_note	Context for reading the explanation

12.5 Explanation Integrity

A backend-blocked transaction must not be described as a TabTransformer decision. Likewise, missing XAI values must not be replaced with invented values in the frontend.

13. Frontend Design and Implementation

13.1 Design Goal

The React frontend presents AttentionPay as a continuous security session: URL security → transaction selection → transaction preview → processing pipeline → final result and explanation.

13.2 Pages / User Views

View	Responsibility
Security Session / URL Check	URL input, processing state, BERT result and explanation
Scenario Selection	Five scenario cards and Stage 2 transaction retrieval
Transaction Flow	Read-only transaction preview, map, pipeline and Simulate/Analyze control
Results	Decision status, security checks, AI probability and XAI when executed

13.3 Main Implemented Components

Area	Components
Layout	Navbar, PageShell
Stage 1	URLInputForm, URLProcessingAnimation, PhishingResultCard, URLTokenHighlighter
Stage 2	ScenarioDeck, DemoTransactionPreview, SimulateButton
Pipeline	ProcessingPipeline
Location	SecurityMap
Results	TransactionResultCard, ExplanationPanel, FeatureContributionChart, LimeContributionView
State/hooks	AppProvider, sessionReducer, useStage1Analysis, useTransactionSimulation, useTransactionProcessing
API	client.js, phishingApi.js, simulationApi.js

13.4 Pipeline States

The frontend uses pending/processing/passed/blocked/skipped states. If a previous gate blocks execution, later stages are marked skipped rather than shown as executed.

13.5 Frontend Integrity Rules

- Never train or run the TabTransformer in the browser.
- Never generate scenario transaction values in the frontend.
- Never hardcode scenario-to-outcome mapping.
- Never claim streamed backend progress for the current synchronous backend.
- Never display or persist the Stage 2 access token unnecessarily.
- Never invent XAI values when the backend explanation is missing.
- Never label model attention/contribution visualization as causal proof.

14. Frontend ↔ Backend Interaction

14.1 URL Check

React sends the submitted URL to POST /api/v1/analyze/url. The returned blocked state determines whether the simulator can be opened. A phishing response keeps the user in Stage 1; a legitimate response reveals Stage 2.

14.2 Scenario Retrieval

React sends only the selected simulation mode. The backend retrieves the transaction and returns a read-only transaction snapshot. The frontend does not modify the transaction values.

14.3 Processing Request

After the user selects Simulate/Analyze, the frontend sends the transaction identifier together with the Stage 2 access token. The backend performs the security checks, feature engineering, AI inference, and XAI as appropriate.

14.4 Response-to-UI Mapping

Response field	Frontend use
transaction_id	Result key and displayed transaction identifier
layer	Indicates layer_1_security or layer_2_fraud_detection
passed	Overall processing-path status
decision	BLOCK or CONTINUE; source of final action state
reason	Primary explanation text
failed_checks	Layer 1 checks that failed
checks	Detailed API route and travel results
explanation_source	backend_rule or tabtransformer_xai
prediction	Fraud / Legitimate when AI executes
fraud_probability / legitimate_probability	Probability cards or gauge
ai_executed	Controls whether AI UI is rendered or skipped
features	Exact 11-feature model vector when AI executes

14.5 Processing State Flow

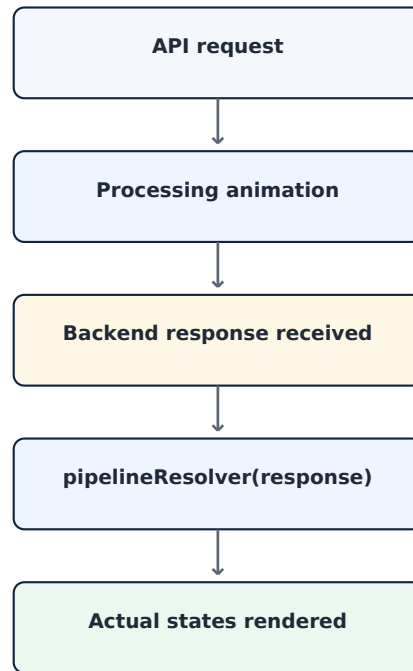


Figure 14.1 — Synchronous frontend processing-state resolution

15. Backend Design — FastAPI

15.1 Backend Responsibilities

Module / service	Responsibility
main.py	Create FastAPI application, CORS, routers, health endpoint
config.py	Application configuration and environment settings
database.py	Database engine/session integration
phishing.py	Stage 1 URL analysis route
simulation.py	Stage 2 transaction retrieval and processing routes
bert_service.py	BERT model loading and URL inference
scenario_service.py	Scenario validation and transaction selection
layer1_security_service.py / security service API	Layer 1 security API route and Impossible Travel checks
feature_engineering.py	Canonical 11-feature extraction and validation
fraud_detection.py	TabTransformer loading, preprocessing, inference and XAI orchestration
explainability/*	BERT, SHAP/LIME and human explanation components
models / schemas	Persistence and API validation structures

15.2 Current API Endpoints

Method	Endpoint	Purpose
GET	/health	Backend health check
POST	/api/v1/analyze/url	BERT phishing URL analysis
POST	/api/v1/simulate/transaction	Retrieve a demo transaction for the selected mode
POST	/api/v1/simulate/transaction/{transaction_id}/process	Run Layer 1, feature engineering, AI and XAI processing

15.3 Stage 2 Access Header

X-Stage2-Access-Token: <temporary-token>

15.4 Stage 2 Processing Order

The implemented processing path is: retrieve transaction → validate Layer 1 security → stop if blocked → feature engineer → TabTransformer inference → XAI → final response. The backend returns the decision source and execution status so the frontend can represent the real path.

16. Database Design

16.1 Purpose

PostgreSQL stores controlled transaction data used by the simulator and provides the persistent source for scenario retrieval. The logical data model separates user/payment URL information, scenario organization, demo transactions, transaction runs, and explanation records.

Entity	Purpose / principal data
users	User identity and type metadata
payment_urls	Submitted URLs, BERT prediction/confidence, phishing state and explanation JSON
scenario_templates	Scenario names and descriptions
demo_transactions	Transaction values, scenario ID, location/time data, expected/actual endpoints, model fields and label
selected_transaction_runs	Each processing execution, backend status/reason, AI execution flag, probabilities, prediction and final s
model_explanations	Explanation type, model name and explanation JSON associated with a transaction run

16.2 Demo Transaction Data

The transaction row is the source of the transaction preview. The internal label is retained for controlled dataset/evaluation purposes and is not treated as the live prediction.

16.3 Seeded Scenario Coverage

The project uses controlled rows covering Normal Transaction, Impossible Travel, API Route Tampering and Behaviour Fraud, with Random Transaction drawing from the complete demonstration pool.

16.4 Transaction Run Integrity

Each execution can be represented as a distinct run so repeated Random Transaction operations can be observed independently for demonstration, debugging and evaluation.

17. Complete System Workflow

17.1 End-to-End Sequence

1. The user opens the React application.
2. The user enters a payment URL.
3. React sends the URL to FastAPI.
4. BERT performs phishing classification.
5. If phishing, the system blocks the flow and shows URL explanation; processing stops.
6. If legitimate, Stage 2 access is enabled.
7. The user selects one of the five simulation modes.
8. FastAPI retrieves a controlled PostgreSQL transaction row.
9. React displays the transaction as a read-only preview.
10. The user starts Simulate/Analyze.
11. Backend runs API Route Integrity and Impossible Travel checks.
12. If a rule fails, the transaction is blocked/held, a rule explanation is returned, and AI is skipped.
13. If checks pass, the backend extracts the canonical 11-feature vector and applies training-consistent preprocessing.
14. TabTransformer produces the fraud/legitimate prediction and probabilities.
15. The appropriate XAI explanation is generated.
16. React displays only the visuals relevant to the actual execution path.

17.2 Random Transaction Workflow

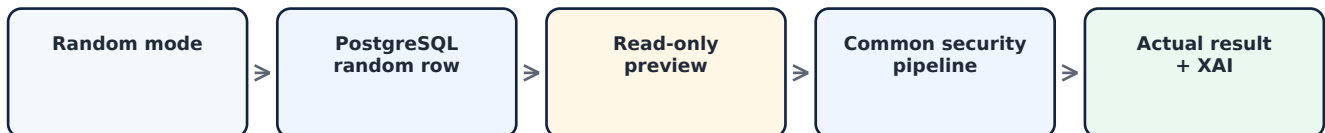


Figure 17.1 — Random Transaction follows the same processing path as every other transaction

17.3 AI Execution Rule

The TabTransformer executes only when the transaction passes the deterministic backend security gates. If Layer 1 blocks, the frontend marks the AI and XAI stages as skipped.

18. Result Dashboard Logic

Decision path	Required UI content
Phishing URL	Blocked status, confidence, highlighted URL tokens and BERT explanation
Impossible Travel	Blocked status, backend rule card, map, distance/time/speed details
API Route Tampering	Blocked status, expected-vs-actual endpoint card and rule explanation
AI Fraud	Fraud decision, probabilities, feature contributions, SHAP/LIME and human explanation
AI Legitimate	Continue/approved status, probabilities, feature contributions and explanation
Random Transaction	Use the actual path of the selected row; never use a generic random-result screen

18.1 Final Decision Logic

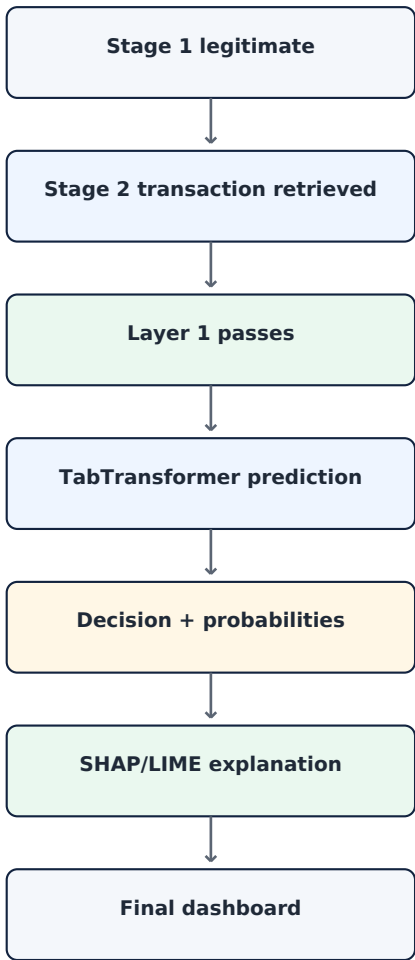


Figure 18.1 — Successful AI-executed result path

18.2 Explainability Source Mapping

Decision source	explanation_source
Stage 1 phishing	bert_url_explanation
Layer 1 block	backend_rule / backend_rule_engine
TabTransformer decision	tabtransformer_xai

19. Repository Structure

```

Attention_Pay/
├── backend/
│   ├── app/
│   ├── dependencies/
│   ├── models/
│   ├── routes/
│   ├── phishing.py
│   ├── simulation.py
│   ├── schemas/
│   ├── services/
│   ├── explainability/
│   ├── utils/
│   ├── config.py
│   ├── database.py
│   ├── main.py
│   └── ml/
│       ├── tabtransformer/
│       ├── artifacts/
│       └── model.py
│   ├── create_tables.py
│   ├── seed_transactions.py
│   ├── requirements.txt
│   └── frontend/
│       ├── public/
│       ├── src/
│       ├── api/
│       ├── app/
│       ├── components/
│       ├── hooks/
│       ├── pages/
│       ├── state/
│       ├── styles/
│       ├── utils/
│       ├── package.json
│       ├── vite.config.js
│       └── model/
│           ├── config.json
│           ├── model.safetensors
│           ├── tokenizer.json
│           └── tokenizer_config.json
│   ├── AttentionPay_SDD.pdf
│   ├── .gitignore
│   └── README.md

```

19.1 Model Artifacts

Location	Artifacts
model/	BERT configuration, safetensors model and tokenizer files
backend/ml/tabtransformer/artifacts/	TabTransformer checkpoint, categorical encoders, scaler, feature configuration and X

19.2 Repository Hygiene

- Only project documentation, source code, model artifacts and required configuration should be retained in the final repository.
- Secrets, virtual environments, dependency caches and generated build folders must remain ignored.
- The root README is the primary project entry point and the SDD is the detailed technical reference.

20. Implementation and Integration Workflow

20.1 Final Implementation Sequence

Step	Implementation activity
1	Project setup and repository inspection
2	Integrate trained BERT model into FastAPI
3	Implement and test URL analysis endpoint
4	Connect Stage 1 block/allow behavior to Stage 2
5	Establish PostgreSQL and SQLAlchemy integration
6	Create and seed controlled transaction scenarios
7	Implement named-scenario and Random Transaction retrieval
8	Implement read-only transaction preview
9	Implement Impossible Travel and API Route Integrity checks
10	Freeze the canonical 11-feature contract and preprocessing rules
11	Prepare dataset and stratified train/validation/test splits
12	Fit preprocessing on training data only
13	Train, evaluate and persist the TabTransformer
14	Integrate model and preprocessing artifacts into backend inference
15	Integrate BERT, rule, SHAP/LIME and human explanation services
16	Build the React security session and pipeline dashboard
17	Connect frontend state to real backend responses
18	Run functional, scenario, model, XAI and UI tests

20.2 Integration Principle

The same model contract is followed from training through inference. The same saved preprocessing artifacts are reused. The frontend consumes the backend contract rather than duplicating security or prediction logic.

20.3 Current Runtime Flow

React → FastAPI → PostgreSQL / ML Services → FastAPI response → pipelineResolver → React result components

21. Testing Strategy

21.1 Functional Testing

Test area	Expected behavior
URL checking	BERT response is returned correctly
Phishing URL	Stage 2 is blocked
Legitimate URL	Stage 2 access is enabled
Scenario selection	Five supported modes are available
Named scenario retrieval	Returned row belongs to requested scenario
Random retrieval	Returned row comes from complete demo pool
Repeated Random runs	Fresh transaction IDs/rows can be returned
Impossible Travel	Impossible movement is blocked
API Route Integrity	Endpoint mismatch is blocked
Feature contract	Exactly 11 features are produced in canonical order
Preprocessing	Inference uses the same fitted artifacts
Layer 1 block	TabTransformer is not executed
Frontend pipeline	States match actual backend execution
Explanation source	Matches the layer that made the decision

21.2 Scenario Testing

Scenario	Expected result path
Normal Transaction	Retrieve → checks pass → TabTransformer → XAI → result
Impossible Travel	Retrieve → travel check fails → block → rule/map explanation → AI skipped
API Route Tampering	Retrieve → endpoint mismatch → block → rule explanation → AI skipped
Behaviour Fraud	Retrieve → checks pass → 11 features → TabTransformer → XAI
Random Transaction	Retrieve any row → common pipeline → actual result

21.3 AI Model Testing

BERT evaluation uses classification metrics including accuracy, precision, recall, F1-score, ROC-AUC and confusion matrix analysis. TabTransformer evaluation uses accuracy, precision, recall, F1-score, ROC-AUC, PR-AUC, confusion matrix, threshold analysis and false-positive/false-negative inspection.

21.4 XAI Testing

- Verify that explanation feature names match the canonical model feature names.
- Verify that SHAP/LIME explanations are generated from the same input used for prediction.
- Verify that Layer 1 blocked transactions do not receive fabricated model explanations.
- Verify that missing explanation payloads result in an explicit skipped/unavailable state rather than invented values.

22. Expected Outputs, Limitations and Future Enhancements

22.1 Expected Outputs

- Legitimate URLs proceed to the transaction simulator.
- Phishing URLs are blocked and explained.
- Named scenarios retrieve matching demo transactions.
- Random Transaction retrieves any demo transaction and follows the common pipeline.
- Backend-blocked transactions show a concrete security reason.
- Eligible transactions reach the AI stage with a reproducible 11-feature vector.
- AI results show probability and explanation.
- The dashboard visibly represents the actual processing path.

22.2 Current Project Limitations

- The system is a controlled academic simulation rather than a production payment-processing platform.
- The transaction database contains demonstration data rather than live banking data.
- The current AI model is constrained by the available training data and feature contract.
- The backend is synchronous; the frontend does not implement real-time streaming of intermediate backend events.
- The model explanation describes influential model inputs and should not be interpreted as guaranteed causal proof.

22.3 Future Enhancements

- Additional behavioural-context features can be introduced through a new coherent model contract and retraining cycle.
- Integration with real banking/payment APIs under appropriate security and compliance controls.
- Live payment gateway and multi-factor authentication integration.
- Permission-based real-time location tracking.
- Graph Neural Network or graph-transformer extensions for relationship-aware fraud detection.
- Continuous monitoring and analyst-review workflows.
- Mobile application and cloud deployment.
- Production-grade behavioural history and account profiling services.

22.4 Model Change Policy

Any future model replacement must preserve the integrity of the training/evaluation protocol: define a new feature contract if required, audit the dataset, use leakage-safe splitting and preprocessing, retrain, evaluate against the current benchmark, preserve inference artifacts, and integrate the new contract consistently across backend and frontend.

23. Conclusion

AttentionPay is a multi-layer explainable payment-security simulator that combines proactive URL security, deterministic transaction security checks, behavioural machine learning, and decision-specific explainability.

The system begins with BERT-based phishing URL detection. A legitimate URL unlocks the controlled transaction simulator. The selected transaction is retrieved from PostgreSQL and displayed as read-only data. Deterministic API Route Integrity and Impossible Travel checks then act as backend security gates. Only transactions that pass these checks are converted into the canonical 11-feature representation and processed by the TabTransformer.

The AI result is accompanied by SHAP/LIME feature contributions and human-readable explanation data. The React dashboard represents the actual execution path, explicitly distinguishing backend rule blocks from AI decisions and marking unexecuted stages as skipped.

The final architectural principle is therefore:

Select data → validate security → prepare the canonical 11-feature vector → apply training-consistent preprocessing → run AI when eligible → explain the actual decision → display the result.

Appendix A — Key API Examples

```
POST /api/v1/analyze/url
{
  "url": "https://example.com"
}

POST /api/v1/simulate/transaction
{
  "scenario": "random_transaction"
}

POST /api/v1/simulate/transaction/{transaction_id}/process
Headers:
X-Stage2-Access-Token: <temporary-token>
```

Appendix B — Local Run Checklist

- Configure PostgreSQL and backend environment settings.
- Activate the Python virtual environment.
- Install backend dependencies.
- Start FastAPI with Uvicorn.
- Install frontend dependencies.
- Start the Vite development server.
- Open the URL security screen.
- Test a legitimate URL and a phishing URL.
- Run all five transaction simulation modes.
- Verify Layer 1 block paths and AI/XAI pass paths.
- Verify Random Transaction and Run Again behavior.

End of Software Design Document